

SplitAlign: Token-Level Cross-Lingual Difference via Aligned LLM Judgments

Hack Apertus Online 2026 · Track 2A UZH SwissGov-RSD · Waterloo Agent Lab · Sharon Basovich · github.com/sharonbasovich/splitalign · Apache-2.0 code / CDLA-Permissive-2.0 predictions / CC-BY-4.0 report

1 Method

SplitAlign decomposes token-level cross-lingual semantic-difference scoring (de/fr/it Swiss administrative document pairs) into auditable stages, preserving the original whitespace-token indices end-to-end so outputs land exactly on the official token label space:

- **Segment** — sentence/paragraph segmentation that records each segment's token offset.
- **Align** — monotone dynamic-programming alignment of cross-lingual sentences with 1:0, 0:1, 1:1, 1:2 and 2:1 (merge) operations, scored by batched Apertus similarity under a **banded candidate policy** ($|i/(n-1) - j/(m-1)| \leq 0.2$ or ± 2 absolute): far-off-diagonal pairs score 0 without an API call — measured 304,968 → 108,260 candidate pairs (35%) on the dev corpus.
- **Judge** — short structured-JSON Apertus judgments on aligned pairs: graded semantic difference 0–5 plus verbatim differing spans, with repair/salvage for malformed responses and explicit handling of repeated tokens, offsets, punctuation, boilerplate and end-of-document omissions.
- **Score** — sentence scores plus span boosts projected onto the exact token arrays; asymmetric additions/omissions get explicit labels.

A whole-document token-annotation prompt serves as the **re-implemented baseline** (same response contract; not the upstream template).

2 Apertus use (required component)

CSCS hosted inference `api.inference.cscs.ch/v1`, model `swiss-ai/Apertus-v1.5-8B` (non-thinking), temperature 0, fixed seed. Apertus performs both the similarity scoring and the graded judgments — no fine-tuning or hidden-state access is assumed. Every call is cached on backend|model|prompt-version|kind|split|item|payload keys and logged redacted (hashes, token counts, latency only). A shared budget caps NEW requests *and* tokens across both methods at the **outbound-attempt boundary**, so retries and failures cannot bypass the cap; identical cache hits are free.

3 Data, licensing, and the held-out firewall

Only `dev/train` and `dev/val` of `ZurichNLP/swissgov-rsd` (CC-BY-4.0 per its Hugging Face card) are fetched — at a pinned commit through an allowlist + fail-closed guard (disallowed splits/paths/symlinks and unmanifested IDs abort). The test split is never fetched, read, or evaluated in this build. Metrics follow the official token-level Spearman convention including its `-1` punctuation filter and `list_to_drop.txt` exclusion (reported both ways). All scoring code is **original Apache-2.0** written from the metric specification — no vendored upstream code (upstream ships no repo-level license).

4 Evaluation — dev/val, 10 documents/language (exploratory)

Method	de	fr	it	Strict macro Spearman
SplitAlign (Apertus-8B)	0.254	0.267	0.224	0.248

Run `20261001T211451Z-8de1aab7`: 30/30 planned items, 97 new requests / 72.6k new tokens under the shared cap. The primary macro is strict — null if any required language is missing or undefined (all three finite here); nonfinite values serialize as JSON null with explicit reasons, never bare NaN. The matched same-ID baseline comparison is pending an authorized bounded run; earlier pre-fix diagnostics are quarantined in `results/runs/legacy-pre-runs/`. Mock-backend outputs (`MOCK-NOT-APERTUS`) are plumbing validations only — never model results.

Honesty note. These are exploratory dev-only figures. No comparative or held-out performance claim is made; the constant-output failure mode of prompt baselines is treated as an informative result, not a success.

5 Reproducibility

Repo `sharonbasovich/splitalign` (public). Root make `run` builds Docker and runs a mock pipeline on a clean checkout; real runs set `SPLITALIGN_BACKEND=apertus + APERTUS_*` env vars (contradictory config fails closed). Every run writes an immutable `results/runs/<ts-id>/` scope — predictions, intended-ID manifest, details, redacted call log, per-mode and matched-ID evals — committed to git. Seed 0 throughout; payload-keyed cache gives bit-identical reruns. A static bilingual evidence viewer (`track_2a/viewer/`) shows aligned spans, per-token heat maps and omissions/additions with real-vs-mock provenance banners.

6 Limitations

Uniform token scores per aligned sentence except reported spans; monotone alignment only (no reordering); banded candidates never score far-off-diagonal pairs; judgments may omit end-of-document differences (handled, logged); held-out integrity is a development guard, not a proof.

7 AI-assistance disclosure & references

Strategy brief and implementation produced by AI assistants (reviewer session + Devin); S. Basovich supplied goals, approvals and credential provisioning. Sources: Hack Apertus Track 2A brief; ZurichNLP/SwissGov-RSD; HF dataset card (CC-BY-4.0); CSCS Inference

API docs; swiss-ai/Apertus-v1.5-8B. Paper prior-work figures (not our measurements): author-prompted LLM ≈ -0.9 macro Spearman, DiffAlign 18.6, sentence-label oracle 56.4.